Profiling Report: LFW Face Verification System

MSML/MSAI 605 – Milestone 4

1. Goal

Measure how the final embedding-based verifier (FaceNet/VGGFace2 → cosine similarity → threshold) behaves on real hardware. The required questions are: (a) which stage dominates latency, (b) how does latency / throughput change with batch size, and (c) is the CPU baseline reproducible from a clean clone.

The system version profiled is the same one the System Card describes (`configs/m3.yaml`, threshold $\theta = 0.351759$). The profiler reproduces the exact preprocessing, embedding, and scoring used at inference time, so the numbers below apply to the deployed pipeline.

2. Measurement Environment (CPU baseline)

- **Host**: Apple Silicon (`arm64`), 8 logical cores; macOS 15.7.4 (`macOS-15.7.4-arm64`). PyTorch 2.2.2, Python 3.12.0.
- **Device**: `cpu` (chosen explicitly so the baseline is comparable across machines without GPU). `torch.get_num_threads() = 8`.
- **Model**: `InceptionResnetV1(pretrained="vggface2")`, weights cached locally / baked into the Docker image.
- **Inputs**: 901 LFW test pairs from `outputs/pairs/test.csv`, 160×160 RGB after resize, normalized to $[-1, 1]$.
- **Tooling**: `scripts/profile_latency.py` (this repo), `time.perf_counter()` wall-clock timing.
- **Run parameters**: 25 distinct pairs for the per-pair stage breakdown (with 3 warm-up iterations), batch sizes $\{1, 2, 4, 8, 16, 32\}$ swept with 3 per-batch warm-up rounds and 5 timed repetitions each.
- **No GPU run included.** The host has no CUDA. MPS is available but is *not* reported as a substitute for the required CPU baseline.

3. Methodology

The profiler is a straight rewrap of the production inference path so the per-stage numbers add up to the end-to-end number. For each stage we time:

- **Preprocessing**: `Image.open + convert("RGB") + resize(160) + normalize`$([-1, 1])$ `+ tensor cast`, per image, summed across the pair.
- **Embedding**: `model.__call__` forward pass per image (single-pair view) or per batch (batch-size sweep), inside `torch.no_grad()`.

- **Scoring**: vectorized `cosine_similarity` from `src/similarity.py` on the resulting embedding rows.
- **End-to-end**: timer started before stage 1, stopped after stage 3, so the sum of stages includes all bookkeeping.

For the batch-size sweep each batch has its own warm-up loop (preprocess + embed `warmup` times) before timing, so model graph caches and tensor allocator are hot at *that* batch size.

Aggregations are mean / median / P95 / min / max across repetitions; for the per-pair breakdown the unit is one pair, for the batch sweep the unit is one batch. A single launch produces `outputs/profiling/per_stage_latency.json`, `outputs/profiling/batch_size_sensitivity.csv`, `outputs/profiling/profiling_environment.json`, and a human-readable `outputs/profiling/profiling_summary.md`.

4. Per-Stage Latency (single-pair, CPU baseline)

Stage	Mean (ms)	Median (ms)	P95 (ms)	Min (ms)	Max (ms)
Preprocessing	7.01	6.76	8.15	6.06	8.51
Embedding (FaceNet)	118.70	116.60	151.68	83.92	229.23
Scoring (cosine)	0.18	0.14	0.37	0.12	0.57
End-to-end	125.88	124.29	158.07	90.65	237.00

Table 1: Per-pair latency, $n = 25$ distinct pairs. Source: `outputs/profiling/per_stage_latency.json`.

Stage share at batch size 1 (mean): embedding $\approx 94.3\%$, preprocessing $\approx 5.6\%$, scoring $< 0.2\%$. Embedding is the unambiguous bottleneck on CPU.

5. Batch-Size Sensitivity (CPU baseline)

Batch	Pre (ms)	Embed (ms)	Score (ms)	E2E (ms)	E2E/pair (ms)	Pairs/s
1	7.60	112.45	0.13	120.18	120.18	8.32
2	14.99	182.33	0.15	197.46	98.73	10.13
4	30.95	347.57	0.16	378.68	94.67	10.56
8	51.93	605.72	0.25	657.90	82.24	12.16
16	116.22	6684.86	0.21	6801.28	425.08	2.35
32	212.29	9663.66	0.55	9876.50	308.64	3.24

Table 2: Latency totals and per-pair amortized cost across batch sizes. Columns: preprocessing total, embedding total, scoring total, end-to-end total, end-to-end per pair, throughput. Source: `outputs/profiling/batch_size_sensitivity.csv`.

Reading the table:

- **Sweet spot at batch 8.** Per-pair end-to-end drops from ≈ 120 ms (batch 1) to ≈ 82 ms (batch 8); throughput rises from ≈ 8.3 pairs/s to ≈ 12.2 pairs/s. Below batch 8 the model is Python-overhead-bound; the model forward pass benefits from larger tensors.

- **Sharp regression at batch 16.** On this 8-core Apple Silicon host the embedding total jumps an order of magnitude (605 ms → 6,685 ms). It is consistent across re-runs and across both `warmup=2` and `warmup=3` settings, and it is concentrated in the embedding stage (preprocessing scales linearly). The most plausible cause is exceeding the per-thread working-set size on macOS Accelerate / OpenMP, possibly compounded by thermal throttling – the model has many conv layers and at batch 16 each forward pass touches a much larger activation tensor than the L2/L3 caches comfortably hold. We report the raw numbers here rather than smooth them away.
- **Batch 32 partially recovers (≈ 309 ms/pair)** but never matches batch 8. Practical guidance from this baseline: *run with batch size 4–8 on CPU*; if a deployment target is GPU, that ceiling will move and should be re-profiled there.
- **Scoring cost is negligible at every size**, so optimizing the embedding stage (smaller backbone, ONNX/quantization, GPU) is the only lever that materially moves end-to-end latency.

6. Stage-by-Stage Interpretation

- **Embedding dominates.** On CPU it accounts for 93–98% of end-to-end latency at every batch size we measured. Any future optimization work should start here.
- **Preprocessing scales linearly** (≈ 7 ms/image regardless of batch). It is bound by PIL decode + numpy cast, so it would benefit from `torchvision` GPU decode if a GPU were available, but it is not the bottleneck today.
- **Scoring is essentially free** (< 0.2% of end-to-end). The vectorized cosine kernel from Milestone 1 stays cheap even at the largest batch.
- **Concurrency** (Milestone 3 load test, 4 workers, 100 single-pair requests): ≈ 22.6 req/s with mean latency ≈ 0.18 s and a tight P95. The 4-thread thread-pool successfully overlaps Python overhead on top of the CPU-bound embedding.

7. Optional GPU Comparison

Not included in this release. The CPU baseline above is the required reference. The host on which this milestone was prepared has no CUDA device, and the MPS backend on Apple Silicon was not used because we did not want to compare a CUDA result to a non-CUDA result in the same table. The profiler accepts a `--device cuda` flag if the next reviewer wants to add GPU numbers; if so, the device, driver, and software stack should be stated alongside the new table and kept clearly separate from the CPU baseline.

8. Reproducing These Numbers

From a clean clone, after the standard environment setup (`python -m venv .venv && pip install -r requirements.txt`):

```
python scripts/profile_latency.py --batch-sizes 1 2 4 8 16 32 \
    --num-single-pairs 25 --warmup 3 --repeat 5
```

This writes the JSON / CSV / Markdown artifacts referenced above into `outputs/profiling/`. Re-running the same command on the same host gives results within the noise band shown by the min/max columns. See `reports/reproducibility_checklist.md` for the full release-level path.